

Big M method:

We assign a big value (infinite) to our slack/surplus variables in order to heavily penalise giving them the final value while we perform our simplex method

Step 1:

Initialise numpy and input functions

```
1  import numpy as np
2  import re
3  ri = lambda: int(input()) # read one int
4  rl = lambda: list(map(int, input().split())) # read line of ints -> list
5  rlf = lambda: list(map(float, input().split())) # read line of ints -> list
6  rm = lambda: map(int, input().split()) # read line of ints -> a, b = rm()
7  rs = lambda: input().strip() # read string (strips newline)
8  ep=1e-9
9  M=1e9
```

Step 2:

Input the number variables, max/min functions and inequalities/equations

```
12  n=int(input("number of variables: "))
13  mode=input("minimise or maximise (min/max): ")
14  flip=-1 if 'max' in mode else 1
15  print("objective coefficients:")
16  cc=rlf()
17  m=int(input("number of constraints: "))
18  print("constraints (coeffs relation rhs per row):")
19  a=[];b=[];r=[]
20  for i in range(m):
21      l=rs().replace('≥','>=').replace('≤','<=')
22      p=re.split(r'(<|=|>|=|<|>)',l)
23      a.append([float(x) for x in p[0].split()])
24      r.append(p[1])
25      b.append(float(p[2]))
```

Step 3:

Add the surplus/slack variables

```
28     t=n
29     sl,su,ar={},{},{}
30     for i in range(m):
31         if r[i]=='<=':
32             sl[i]=t;t+=1
33         elif r[i]=='>=':
34             su[i]=t;t+=1
35             ar[i]=t;t+=1
36         else:
37             ar[i]=t;t+=1
38     mat=np.zeros((m,t))
39     for i in range(m):
40         mat[i,:n]=a[i]
41         if r[i]=='<=':
42             mat[i,sl[i]]=1
43         elif r[i]=='>=':
44             mat[i,su[i]]=-1
45             mat[i,ar[i]]=1
46         elif r[i]=='=':
47             mat[i,ar[i]]=1
```

Step 4:

Make the table

```
48     bb=np.array(b,float)
49     price=[0.0]*t
50     for j in range(n):
51         price[j]=cc[j]*flip
52     for j in ar.values():
53         price[j]=M
54     basis=[sl[i] if r[i]=='<=' else ar[i] for i in range(m)]
```

Step 5:

The main loop iterating to find a case which is optimal

```

57     for _ in range(1000):
58         B=mat[:,basis]
59         xb=np.linalg.solve(B,bb)
60         y=np.linalg.solve(B.T,[price[j] for j in basis])
61         red=[price[j]-mat[:,j]@y for j in range(t)]
62         ent=None
63         for j in range(t):
64             if j not in basis and red[j]<=-ep:
65                 ent=j;break
66         if ent==None:
67             xx=np.zeros(t)
68             for k,j in enumerate(basis):
69                 xx[j]=xb[k]
70             break
71         d=np.linalg.solve(B,mat[:,ent])
72         if max(d)<=ep:
73             print("The problem is unbounded.")
74             quit()
75         best=-1;br=float('inf')
76         for i in range(m):
77             if d[i]>ep:
78                 rr=xb[i]/d[i]
79                 if rr<br:
80                     br=rr;best=i
81         basis[best]=ent

```

Step 6:

Confirm answer and print

```

83     ok=True
84     if 'xx' in locals():
85         for j in ar.values():
86             if xx[j]>ep:
87                 ok=False
88     if 'xx' not in locals():
89         print("The problem is unbounded.")
90     elif not ok:
91         print("The problem is infeasible.")
92     else:
93         print("Optimal solution found.")
94         for i in range(n):
95             print("x%d = %.4f"%(i+1,xx[i]))
96         print("Objective value: %.4f"%sum(cc[i]*xx[i] for i in range(n)))

```

Test case:

```
number of variables: 4
minimise or maximise (min/max): max
objective coefficients:
40 30 35 20
number of constraints: 5
constraints (coeffs relation rhs per row):
1 1 1 1 <= 100
2 1 3 1 <= 150
1 3 2 1 <= 120
1 0 2 0 = 40
0 1 0 3 >= 20
```

Gives:

```
Optimal solution found.
x1 = 40.0000
x2 = 10.0000
x3 = 0.0000
x4 = 50.0000
Objective value: 2900.0000
```

Vogel method:

We assign penalties as the difference of the least 2 values in each row/column and allot supply/demand in the row/column with the highest penalty to a cell with least cost

Step 1:

Initialise numpy and input functions

```
1  import numpy as np
2  ri = lambda: int(input()) # read one int
3  rl = lambda: list(map(int, input().split())) # read line of ints -> list
4  rlf = lambda: list(map(float, input().split())) # read line of ints -> list
5  rm = lambda: map(int, input().split()) # read line of ints -> a, b = rm()
6  rs = lambda: input().strip() # read string (strips newline)
```

Step 2:

Take input of cost matrix. Then of supply and demand vectors

```
8  # cost matrix
9  rows = int(input("rows: "))
10 cols = int(input("cols: "))
11
12 print("cost matrix:")
13 cost = [rlf() for _ in range(rows)]
14
15 # supply and demand matrix
16 print("supply vector:")
17 supply = rlf()
18 print("demand matrix:")
19 demand = rlf()
20
21 # balance
22 t_supply=sum(supply)
23 t_demand=sum(demand)
```

Step 3:

Add dummy row/column if supply!=demand

```

25 # Add dummy row/column as needed
26 if t_supply>t_demand:
27     print(1)
28     for i in range(rows):
29         cost[i].append(1e5)
30     demand.append(t_supply-t_demand)
31     cols+=1
32 elif t_demand>t_supply:
33     print(2)
34     temp=[1e5]*cols
35     supply.append(t_demand-t_supply)
36     cost.append(temp)
37     rows+=1

```

Step 4:

Initlaise matrix to perform calculations on

*cc is for vogel+modi method

```

38 tt=t_demand=t_supply
39 cost=np.array(cost)
40 ans=0
41
42 table = np.zeros((rows, cols))
43 cc=cost.copy()

```

Step 5:

Calculate row and column penalty

```

46     def diff(cost):
47         row=[]
48         col=[]
49         for i in range(rows):
50             min1,min2=1000,1000
51             for j in cost[i]:
52                 if j<min1:
53                     min2=min1
54                     min1=j
55                 elif j<min2:
56                     min2=j
57             row.append(min2-min1)
58         for i in range(cols):
59             min1,min2=1000,1000
60             for t in range(rows):
61                 j=cost[t][i]
62                 if j<min1:
63                     min2=min1
64                     min1=j
65                 elif j<min2:
66                     min2=j
67             col.append(min2-min1)
68         return row,col

```

Step 6:

If maximum value appears in a row then find that value, allot the smallest value in the row then

```

71 while tt>0:
72     row,col=diff(cost)
73     max1=max(row)
74     max2=max(col)
75     if max1>=max2:
76         for i in range(rows):
77             if row[i]!=max1:
78                 continue
79             minn=min(cost[i])
80             for j in range(cols):
81                 if cost[i][j]!=minn:
82                     continue
83                 table[i][j]=min(supply[i],demand[j])
84                 supply[i]-=table[i][j]
85                 demand[j]-=table[i][j]
86                 ans+=table[i][j]*cost[i][j]
87                 if demand[j]==0:
88                     for k in range(rows):
89                         cost[k][j]=1000
90                 else:
91                     for k in range(cols):
92                         cost[i][k]=1000
93             break
94         break

```

Else do the same in column

```

95     else:
96         for j in range(cols):
97             if col[j]!=max2:
98                 continue
99             minn=min([cost[i][j] for i in range(rows)])
100             for i in range(rows):
101                 if cost[i][j]!=minn:
102                     continue
103                 table[i][j]=min(supply[i],demand[j])
104                 supply[i]-=table[i][j]
105                 demand[j]-=table[i][j]
106                 ans+=table[i][j]*cost[i][j]
107                 if demand[j]==0:
108                     for k in range(rows):
109                         cost[k][j]=1000
110                 else:
111                     for k in range(cols):
112                         cost[i][k]=1000
113             break
114         break

```

Step 7:

Break if both row and column dont have another value left and then print

```
115         if max1==0.0 and max2==0.0:
116             break
117         tt-=table[i][j]
118
119     print(table)
120     print("cost:",ans)
```

Test:

Input:

```
rows: 3
cols: 4
cost matrix:
3 1 7 4
2 6 5 9
8 3 3 2
supply vector:
300 400 500
demand matrix:
250 350 400 200
```

Output:

```
[[ 0. 300.  0.  0.]
 [250.  0. 150.  0.]
 [ 0.  50. 250. 200.]]
cost: 2850.0
```

MODI method:

We assign a big value (infinite) to our slack/surplus variables in order to heavily penalise giving them the final value while we perform our simplex method

Step 8:

Find the row/column with the most allocations

```
123 def uv_init():
124     seenu=[0 for _ in range(rows)]
125     u=[0 for _ in range(rows)]
126     seenv=[0 for _ in range(cols)]
127     v=[0 for _ in range(cols)]
128
129     mx,index=0,0
130     for i in range(rows):
131         curr_mx=sum([1 for j in range(cols) if table[i][j]>0])
132         if curr_mx>mx:
133             mx,index=curr_mx,i
```

Step 9:

Define u v values for each row/column

```
135 def recur(i,j,isrow):
136     if isrow:
137         for c in range(cols):
138             if seenv[c]==0 and table[i][c]>0:
139                 seenv[c]=1
140                 v[c]=cc[i][c]-u[i]
141                 recur(i,c,0)
142     else:
143         for r in range(rows):
144             if seenu[r]==0 and table[r][j]>0:
145                 seenu[r]=1
146                 u[r]=cc[r][j]-v[j]
147                 recur(r,j,1)
148
149     seenu[index]=1
150     recur(index,0,1)
```

Step 10:

Define the penalty matrix as $u[i]+v[j]-cost[i][j]$

```

152     maxp=0
153     ii,jj=-1,-1
154     for i in range(rows):
155         for j in range(cols):
156             if table[i][j]==0:
157                 pen=u[i]+v[j]-cc[i][j]
158                 if pen>maxp:
159                     maxp=pen
160                     ii,jj=i,j

```

Step 11:

If max penalty is more than 0 we need to allocate

```

162     if maxp <= 0.001:
163         return False
164
165     nodes=[]
166     for i in range(rows):
167         for j in range(cols):
168             if table[i][j]>0:
169                 nodes.append((i,j))
170     nodes.append((ii,jj))

```

Step 12:

Perform dfs to find the cycle

```

172  ✓ def dfs(r,c,path,horizontal):
173  ✓     if len(path)>3 and (r,c) == (ii,jj):
174         return path
175  ✓     for nr,nc in nodes:
176  ✓         if (nr,nc) in path and (nr,nc) != (ii,jj):
177             continue
178  ✓         if horizontal and nr==r and nc!=c:
179             res=dfs(nr,nc,path+[(nr,nc)],False)
180             if res: return res
181  ✓         elif not horizontal and nc==c and nr!=r:
182             res=dfs(nr,nc,path+[(nr,nc)],True)
183             if res: return res
184         return None

```

Step 13:

Reduce the adjacent corners while giving their values to the max cell and opposite cell

```
186     loop_path=dfs(ii,jj,[(ii,jj)],True) or dfs(ii,jj,[(ii,jj)],False)
187     if loop_path:
188         loop_path = loop_path[:-1] # strip duplicate start node
189     else:
190         return False
191
192     minus_cells=[loop_path[k] for k in range(1,len(loop_path),2)]
193     theta=min([table[r][c] for r,c in minus_cells])
194
195     for k in range(len(loop_path)):
196         r,c=loop_path[k]
197         if k%2==0:
198             table[r][c]+=theta
199         else:
200             table[r][c]-=theta
201     return True
```

Step 14:

Print

```
206     print("\nFinal:")
207     print(table)
208
209     ans=0
210     for i in range(rows):
211         for j in range(cols):
212             ans+=table[i][j]*cc[i][j]
213     print("Cost:",ans)
```

Test:

Input

```
rows: 3
cols: 4
cost matrix:
3 2 7 6
7 5 2 3
2 5 4 5
supply vector:
50 60 25
demand matrix:
60 40 20 15
```

Vogel output:

```
[[10. 40.  0.  0.]
 [25.  0. 20. 15.]
 [25.  0.  0.  0.]]
cost: 420.0
```

vogel + modi method:

```
[[35. 15.  0.  0.]
 [ 0. 25. 20. 15.]
 [25.  0.  0.  0.]]
Cost: 395.0
```

*The cost goes down by a small amount